

# Data Analytics

## Assignment 2: Community Detection

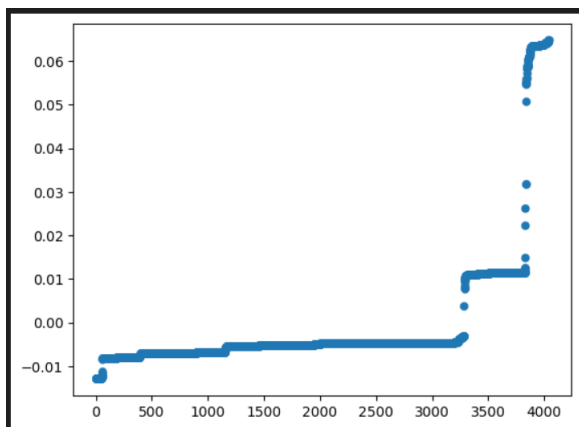
(Sachin Tanwar, MTech AI, SR: 26120)

**Note:** The visualization component has been added within each method for decomposition, hence time present within report for running decomposition vs running them and finding may differ because of visualization component, as time was recorded for only decomposition part of methods.

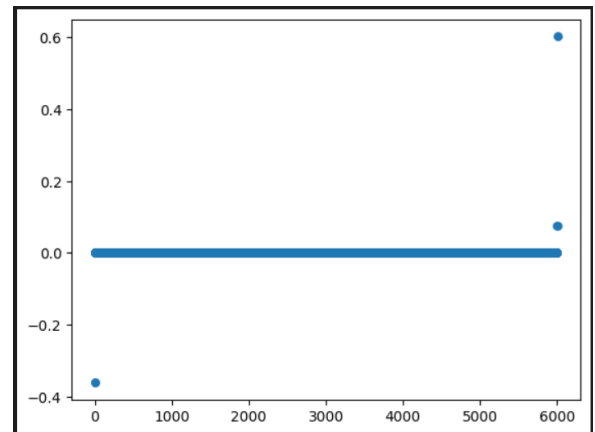
**Ques 1:** Run one iteration of the Spectral decomposition technique. Plot the sorted Fiedler vector, the associated adjacency matrix and the graph partition.

**Ans :** Required Plots and Diagrams are given below for both the data with 1 iteration of spectral algorithm. We got 2 communities after the first iteration, which is quite evident and trivial from what we already know for the first iteration.

### A. Sorted Fiedler Vectors for first Iteration



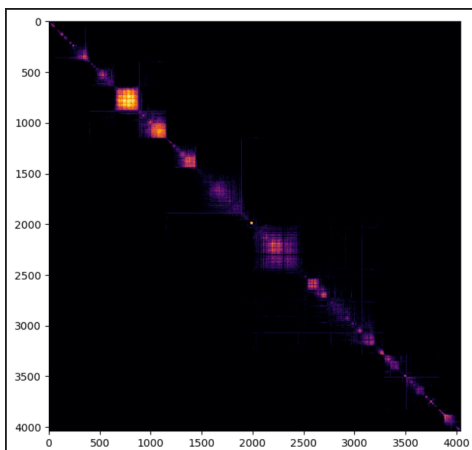
a) Facebook Data



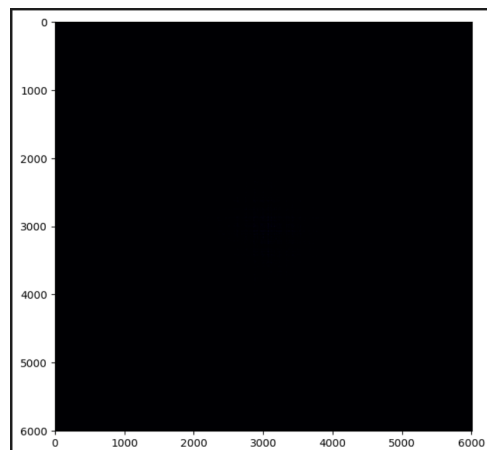
b) Bitcoin Data

### B. Associated Adjacency Matrix for first Iteration

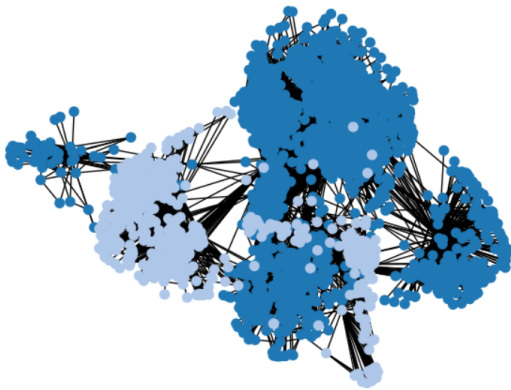
a) Facebook Data



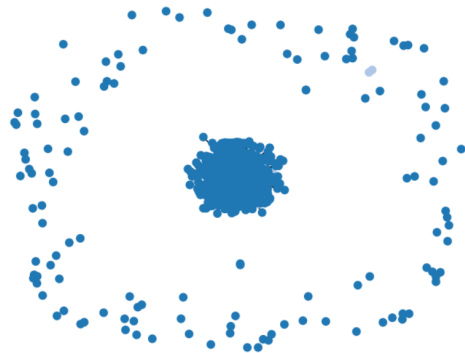
b) Bitcoin Data



### C. Associated Graph Partition for first Iteration



a) Facebook Data



b) Bitcoin Data

**Ques 2:** Come up with an automated algorithm to determine the right set of communities using the spectral decomposition method. What would be your stopping criterion?

Ans: As discussed and hinted in the classroom discussion, to create an automated algorithm we look for two questions :

- a. How can we go for a favorable community partition by looking at fiedler vectors
- b. Minimum how many member can be there in partition

And as mentioned we answer the above question based on a few observations and assumptions and create an iterative algorithm for the purpose. Hence we put the following **stopping criteria**:

1. **maxGap** < **a x avgGap** i.e. if maximum gap ('maxGap') among continuous values of fiedler vector must be less than product of a constant ('a') and average gap ('avgGap') of the consecutive values within same fiedler vector. Value of a is decided on the basis of observation by noting and comparing maxGap and avgGap values for vector
2. To limit the **minimum community size** we follow following approaches:
  - a. If community size is less than **100**, don't partition
  - b. If there are strong evidence for partitioning but after partition any of the **resulting partition size is less than 20**, then avoid the partition created

#### **Algorithm:**

Input: Edge\_List

Output: Partition

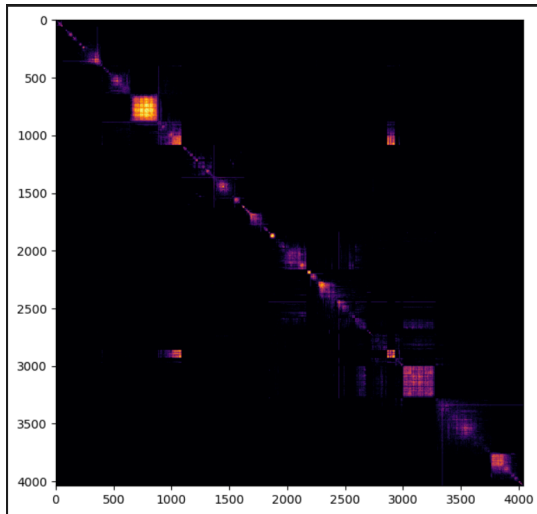
1. Create the Adjacency matrix
2. Find Laplacian Matrix
3. Find Fiedler vector
4. Look for potential Community partition and Evaluate (using above mentioned stopping criteria) each partition
5. If Community is valid, add to partition.
  - a. for each valid partition, go to step 1, to look for further sub-partitions

6. If invalid partition skip to next subpartition within partition list
7. Repeat until the complete partition list is traversed.

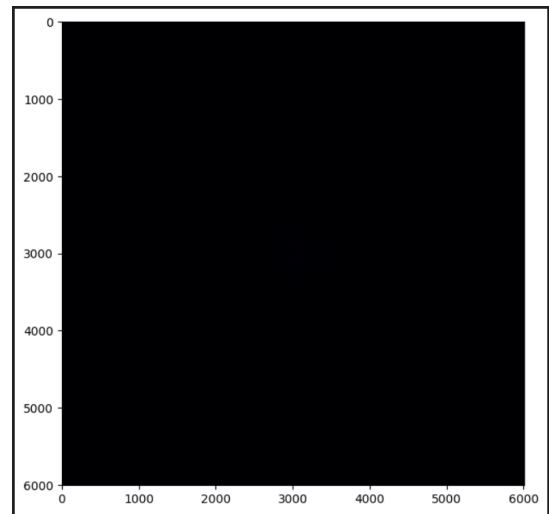
**Ques 3:** Plot the associated adjacency matrix sorted by associated sorted sub-graph Fiedler vectors. Visualize and show the graph that you obtained.

Ans: The Adjacency matrix sorted by associated sorted sub graph Fiedler vectors and Graph visualization is given below:

#### A. Adjacency Matrix

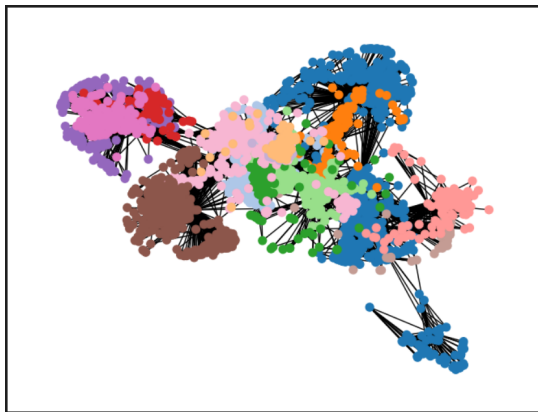


a) Facebook Data

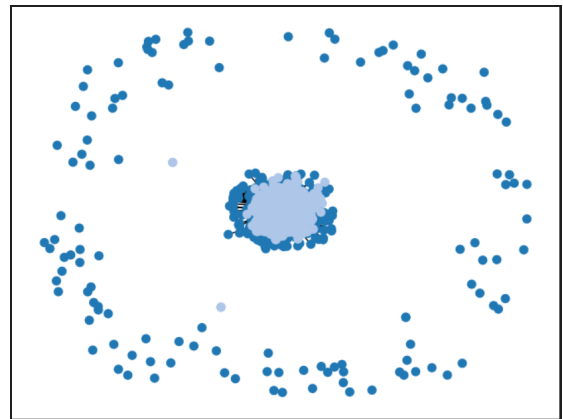


b) Bitcoin Data

#### B. Graph Visualization



a) Facebook Data



b) Bitcoin Data

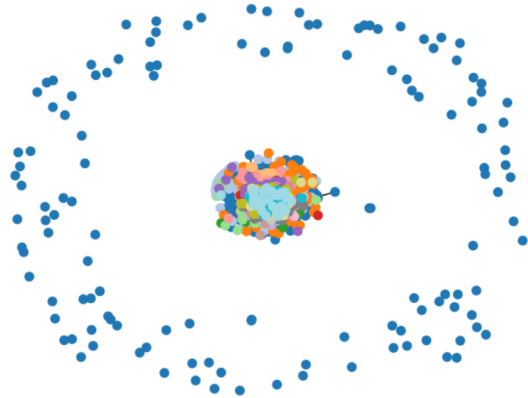
**Observation:** We got **14 communities for Facebook Data** and **2 communities for Bitcoin data** using Spectral Decomposition.

**Ques 4:** Do the same for the Louvain algorithm. Show the communities you get after one iteration.

Ans: Required Graph Visualization and Communities using Louvain Algorithm is shown below:



a) Facebook Data



b) Bitcoin Data

**Observation:** We got **101 communities** and **455 communities** respectively for **Facebook** and **Bitcoin** data using **Louvain Algorithm**

**Ques 5:** How would you pick the best decomposition of nodes into communities?

Ans:

For **Spectral Algorithm**:

- Good enough jump value in the fiedler vector (based on certain threshold, after comparing maximum jump with average jump, as mentioned in algorithm)
- Community size, must not be less than 20 after decomposition
- Current community can only be decomposed, if the number of nodes within it is greater than 100.

For **Louvain Algorithm**:

- Modularity values, in order to maximize it, by rearranging nodes into different communities we stop, when we find that rearranging nodes from one community to another cannot improve the quality of the community structure any further.

We could use and experiment with different criteria as well in addition to those given above like Purity, Normalized Cut and Conductance, Coverage, Silhouette Score. We could use modularity with spectral and fiedler vector concepts to evaluate within Lovain as well.

**Ques 6:** What was the running time of the Spectral decomposition algorithm versus the Louvain algorithm on the data sets you were given?

Ans: Running time for both the algorithms over both the data given are shown below, **values are in seconds**.

|               | Facebook | Bitcoin |
|---------------|----------|---------|
| Spectral Algo | 110.58   | 224.99  |
| Louvain Algo  | 32.12    | 47.51   |

Observations: **Spectral Algorithm** take nearly **4X to 5X** as compared to **Louvain Algorithm**, and any algorithm takes **more time** in finding communities within **Bitcoin data**.

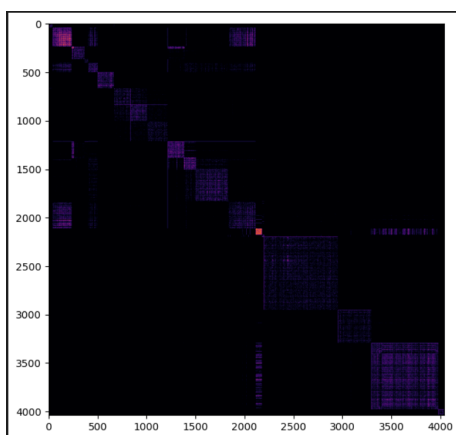
**Ques 7:** In your opinion which algorithm gave rise to better communities, why?

Ans: For better comparison we are going with Facebook data:

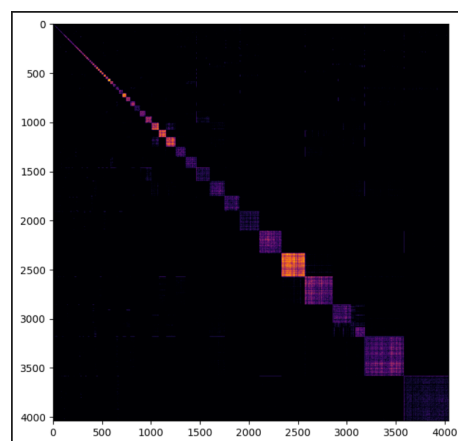
- If we talk in terms of **computational time efficiency**, it's quite evident that **Louvain** algorithm does better, taking lesser time for finding community
- Louvain** algorithm, clearly shows **better community structures**, evident by plotting the final Adjacency Matrix (sorted by "**createSortedAdjMat**" based on community size) after getting partitions from both spectral and louvain algorithm
- Within the Community**, the **Spectral Algorithm fails** to provide **good representation** for their ordering, whereas in the Louvain algorithm, it solves this problem beautifully.
- Though we get good **community size resembling actual communities** with the **spectral algorithm** as compared to Louvain, but that is because we only ran one iteration for the Louvain algorithm, it could perform better if it could run its multiple iteration.
- Stopping criteria** for **Louvain algorithms** seems much **reasonable** based on **modularity** than **community size** and **jumps in the values of fiedler vector** for **spectral algorithm**

Hence with above mentioned points, **Louvain Algorithm** seemed much **better** for community detection problems.

**Sorted Matrix using "**createSortedAdjMat**" method** after getting partition from **Spectral and Louvain Algorithm** on **Facebook** data.



a) Spectral Algorithm



b) Louvain Algorithm